Build and Release a Service with AI Assistance

Individual capstone | Suggested duration 2.5–3 hours with prepared lab resources

Create a small Java or Python service from scratch using GitHub Copilot. Use Azure DevOps to manage its journey from work-item planning to an approved Azure App Service release, supported by an AI-assisted readiness report. Each participant owns and demonstrates their project.

Scenario and application requirements

Build an Order Management Service using Python with Flask/FastAPI or Java with Spring Boot. A User Service or similar small application is also acceptable if it demonstrates equivalent functionality and delivery activities. In-memory storage is sufficient.

Capability	Acceptance criteria
Create and retrieve orders	Accept product name, quantity and unit price; return an ID and calculated total. Retrieve an existing order and return an appropriate not-found response for a missing ID.
Validate and calculate	Reject missing product names, non-positive quantities and non-positive prices. Apply a 10% discount when the subtotal is 1,000 or more; no discount below 1,000.
Check health	Provide a health endpoint for post-deployment verification.

1 Plan the work in Azure Boards

Create a User Story in an Agile project, or the equivalent work-item type in the selected process. Example: As a customer, I want to create and retrieve orders with the correct total and applicable discount. Record the acceptance criteria above.

Create linked tasks for application creation, validation/discount logic, AI code review, test generation, CI/CD, release-readiness reporting, approval/deployment and final verification. Link the branch, commits and PR. Update task status as work progresses; close the story only after acceptance and deployment verification.

2 Create update and review the code

Use Copilot in VS Code to scaffold the project and dependencies. Read, run and correct the output. On a linked feature branch, implement the discount change and open an Azure DevOps PR. Ask Copilot to review calculation, validation, edge cases and consistency. Record accepted/rejected findings, reasons and verification in the PR. An IDE review is sufficient for the core exercise.

3 Use meaningful commit messages

Make small reviewable commits. Describe each change and include its actual Azure Boards work-item reference. Replace the sample IDs below; avoid messages such as changes, updated code or final fix.

Purpose	Example commit message
Feature	feat: add order creation endpoint #123
Business rule	feat: apply discount for subtotals of 1000 or more #123
Tests	test: cover discount boundary and invalid inputs #124
Correction	fix: include subtotal of 1000 in discount eligibility #123
Pipeline and documentation	ci: add test and packaging stages #125 docs: add App Service verification instructions #126

4 Generate tests and prove failure blocking

Use Copilot to generate tests, then inspect assertions. Cover subtotals below, equal to and above 1,000; invalid quantity/price; missing product name; and existing/missing order retrieval. Change >= 1000 to > 1000 deliberately, show the failed boundary test blocking progression, then correct it and rerun successfully.

5 Build a multi stage Azure Pipeline

Stage	Required action and evidence
Build	Restore dependencies and validate/build the selected revision.
Test	Run automated tests and publish results, including after failure. Failed tests block packaging and later progression.
Package	Publish an identifiable versioned ZIP or JAR-based deployment artifact.
Release Readiness	Use the supplied script to gather Azure DevOps evidence and call Azure OpenAI; publish the report and draft release notes.
Approval	Pause for the instructor/designated reviewer to record approval or rejection after reviewing the evidence.
Deploy	Deploy the approved artifact to Azure App Service using the authorised service connection.
Verify	Check application health, confirm the expected version and exercise a core API operation. Retain the results.

Deploy the same packaged artifact without rebuilding it in the deployment stage. Use the trainer-provided Java or Python deployment example and tested approval configuration. Each participant owns the full implementation; the instructor/designated reviewer supplies the release approval.

6 Produce an AI assisted readiness report

Use the prepared Azure DevOps/Azure OpenAI integration script. Configure it for your project and retain these report sections:

- Release identity: work item, PR, source revision, pipeline run, artifact version and intended target.
- Measured evidence: build outcome, test counts/failures and package availability, with links to the relevant run.
- AI contribution: change summary, behaviour affected, evidence-grounded risk observations, uncertainty and draft release notes.
- Human decision: go/no-go or hold, reviewer and rationale. Attach health and functional verification after deployment.

Decision rules: failed mandatory checks mean no-go. Missing evidence or unavailable AI analysis means hold for review. AI cannot override a failed control or approve deployment. Review its statements against actual evidence.

7 Demonstrate and submit your project

Show the story and tasks from planning to completion; linked meaningful commits and PR; one verified Copilot finding; meaningful generated tests; failed and corrected pipeline runs; the readiness report and approval; and the deployed App Service with verification. Submit repository, work-item, PR and run links, the report and the deployed URL.

Optional enhancements

Bicep provisioning; security scans and extra quality gates; deployment slots, rollback or progressive deployment; persistent storage, authentication or UI; repository-native automated AI review comments; additional coverage reporting or operational monitoring.

Assessment criteria

Criterion	Weight
Work-item planning traceability and meaningful commits	15%
Working application and correct business change	20%
Verified AI review and meaningful generated tests	20%
Multi-stage pipeline failure blocking and App Service deployment	25%
AI readiness report human decision and deployment verification	20%

Lab preparation

IT/trainer provides each participant a repository or isolated project area, App Service target, authorised service connection, working agent capacity and approved AI access. Provide the release-readiness starter script, approval YAML and runtime-specific deployment examples in advance. Use approved credential handling and synthetic data.